

multi_brains Framework Source Code Analysis

multi_brains Framework Source Code Analysis

1. Course Content
2. Source Package Structure
 - 2.1 Package File Structure
3. Text Interaction Feature
 - 3.1 ROS Topic Communication
 - 3.2 Terminal Control Commands
4. Agent Core Node
 - 4.1 Action Registry
 - 4.2 Text Input Callback
 - 4.3 Dify Feedback Loop
 - 4.4 Box-Selection Layer and Tracking Actions
5. Action Execution Feature
6. Preemption and Stop Feature
 - 6.1 New Instruction Preemption
 - 6.2 Worker Thread Expiry Determination
 - 6.3 Stop Action

1. Course Content

- Analyze the core functionality of the multi_brains package architecture.
- The current version uses terminal text interaction as the default entry point, replacing the speech recognition and voice wake-up flow with `terminal_client.py`.
- In later courses, each newly introduced action function will be explained individually.

Tip

- This course is a pure program analysis course for users who want an in-depth understanding of the features. If you only need to experience the features, you can skip this course.

2. Source Package Structure

2.1 Package File Structure

```
├─ config
│   ├── color_hsv.txt
│   ├── llm_agent_config.yaml
│   ├── map_mapping.yaml
│   └─ README.md
├─ language
│   ├── en.yaml
│   └─ zh.yaml
├─ launch
│   └─ llm_agent_control.launch.py
```

```

|   └─ terminal_client.launch.py
└─ multi_brains
|   └─ clients
|       │   └─ image_preview.py
|       │   └─ terminal_client.py
|       └─ controllers
|           │   └─ follow_line_controller.py
|           │   └─ kcf_tracker.py
|           │   └─ motion_controller.py
|           │   └─ navigation_controller.py
|           │   └─ ptz_controller.py
|           └─ core
|               │   └─ action_registry.py
|               │   └─ command_state.py
|               │   └─ feedback_protocol.py
|               │   └─ hardware_profile.py
|               └─ __init__.py
|               └─ llm_agent_node.py
|               └─ services
|                   │   └─ dify_coordinator.py
|                   │   └─ dify_client.py
|                   └─ utils
└─ package.xml
└─ resource
    └─ multi_brains
└─ setup.cfg
└─ setup.py

```

- config

Configuration folder, stores configuration file templates. The current core configuration file is `llm_agent_config.yaml`, which contains parameters such as the Dify address, action feedback loop, navigation, line following, KCF tracking, camera, pan-tilt, and chassis safety limits.

- clients/terminal_client.py

Terminal text interaction client, responsible for reading user input and publishing it to `/llm_text_input`, while subscribing to `/llm_text_output` to display the robot's progress, replies, and action list.

- llm_agent_node.py

The agent core node, responsible for loading configuration, connecting to Dify, parsing actions, executing actions, and feeding action results and images back to Dify.

- services/dify_coordinator.py

Dify access and response parsing module, encapsulating SSE requests, file uploads, conversation_id management, and action JSON parsing.

- controllers

Specific hardware action controllers. The chassis, navigation, pan-tilt, line following, and KCF following are split into separate files for easier maintenance and extension.

- core

Pure logic helper modules, including action descriptions, feedback protocols, hardware model capability

determination, and single-command state.

- docs/prompts

Prompt templates for each layer of AI nodes in the Dify workflow. Currently includes the decision layer, execution layer, and box-selection layer, where the box-selection layer is specifically responsible for target bbox output in visual tracking tasks.

- launch

ROS2 launch file directory. `llm_agent_control.launch.py` starts the core node, and `terminal_client.launch.py` starts the terminal text client.

- language

Stores log files in different languages.

3. Text Interaction Feature

The current package no longer uses speech recognition and voice wake-up as the default entry point, but instead uses `terminal_client.py` for text interaction. This does not depend on microphones, wake words, or ASR/TTS services, making it more suitable for development debugging and course reproduction.

Source code path:

```
~/microros_ws/src/multi_brains/multi_brains/clients/terminal_client.py
```

3.1 ROS Topic Communication

The `TerminalClient` node does two things:

1. Creates a `/llm_text_input` publisher, wrapping terminal input as `LlmTextRequest`.
2. Creates a `/llm_text_output` subscriber, receiving `LlmTextResponse` and displaying progress, final replies, and executed actions.

```
class TerminalClient(Node):
    def __init__(self):
        super().__init__("terminal_client")
        self.text_pub = self.create_publisher(LlmTextRequest, "/llm_text_input", 10)
        self.text_sub = self.create_subscription(
            LlmTextResponse, "/llm_text_output", self.response_callback, 10
        )

    def send_text(self, text: str):
        msg = LlmTextRequest()
        msg.text = text
        msg.is_feedback = False
        self.text_pub.publish(msg)
```

3.2 Terminal Control Commands

Besides natural language instructions, the terminal client also has several built-in local control commands:

Command	Function
<code>reset</code>	Sends <code>__reset_conversation__</code> to the core node, resetting the Dify conversation context
<code>c1ear</code>	Clears the screen and redisplay the help banner
<code>help</code>	Displays terminal client help
<code>exit / quit / q</code>	Exits the terminal client

Startup method:

```
ros2 run multi_brains terminal_client
```

4. Agent Core Node

Source code path:

```
~/microros_ws/src/multi_brains/multi_brains/llm_agent_node.py
```

The core node `LlmAgentNode` is the main scheduler of the current package, mainly responsible for:

1. Loading `llm_agent_config.yaml` and allowing environment variables such as `DIFY_API_KEY` and `CAR_TYPE` to override the configuration.
2. Subscribing to `/llm_text_input` to receive terminal text requests.
3. Publishing `/llm_text_output` to return decision progress, final replies, and action lists to the terminal.
4. Subscribing to `/camera/image_raw`, caching the latest camera image for use by `seewhat()` and `observe_surroundings()`.
5. Calling `DifyCoordinator` to complete Dify decision making, box-selection layer result reading, response parsing, and multi-round feedback loops.
6. Distributing the action strings output by Dify to the corresponding controllers through the action registry.

4.1 Action Registry

The action registry defines the action functions that Dify can call, as well as the local handler function corresponding to each action.

```
def init_action_registry(self):
    self.action_registry = {
        "navigation": self.navigation.execute_navigation,
        "follow_line": self.execute_follow_line,
```

```

"track_kcf": self.kcf.execute_track_kcf,
"seewhat": self.execute_seewhat,
"move_forward": self.motion.execute_move_forward,
"move_backward": self.motion.execute_move_backward,
"turn_left": self.motion.execute_turn_left,
"turn_right": self.motion.execute_turn_right,
"stop": self.execute_stop,
"servo1_move": self.ptz.execute_servo1_move,
"servo2_move": self.ptz.execute_servo2_move,
"servo_init": self.ptz.execute_servo_init,
"servo_shake": self.ptz.execute_servo_shake,
"servo_nod": self.ptz.execute_servo_nod,
"get_current_pose": self.navigation.get_current_pose,
"observe_surroundings": self.execute_observe_surroundings,
"finishtask": self.execute_finish_task,
}

```

4.2 Text Input Callback

When the terminal client publishes a new text, `text_callback` creates a new command number. If actions such as navigation, line following, or KCF following are currently running, the new instruction takes priority and the core node cancels the current actions as soon as possible.

```

def text_callback(self, msg: LlmTextRequest):
    if msg.is_feedback:
        return

    user_text = msg.text.strip()
    if not user_text:
        return

    with self.command_lock:
        self.current_command_id += 1
        command_id = self.current_command_id

    if self.is_action_running() or self.kcf.kcf_tracking_active:
        self.cancel_current_action(
            recenter_camera=self._is_stop_user_command(user_text)
        )

    if user_text == "__reset_conversation__":
        self.dify.reset_conversation_id()
        self.publish_latest_response(command_id, "conversation context reset", "[]", "")
        return

    self.worker_thread = threading.Thread(
        target=self.process_text_command,
        args=(command_id, user_text),
        daemon=True,
    )
    self.worker_thread.start()

```

4.3 Dify Feedback Loop

`process_text_command` is the core flow. It sends the user text to Dify, parses the actions returned by Dify, executes only the first action of the current round, and then feeds the action success/failure, images, or multi-directional images back to the same Dify conversation, until Dify outputs `finishtask()` or the maximum feedback round is reached.

The simplified flow is as follows:

Terminal input

- > Dify decision/execution layer returns response + actions
- > In tracking tasks, the Dify box-selection layer outputs `bbox_OUT` based on the image
- > `llm_agent_node` parses the actions
- > Executes the action
- > Generates robot feedback
- > Requests Dify again with the feedback
- > Dify decides the next action or ends the task

Core function explanations:

- `self.dify.call_agent(...)`: requests the Dify application.
- `self.dify.parse_response(...)`: parses the reply text and action list.
- `select_round_action(actions)`: selects only one executable action per round, ensuring the "execute -> feedback -> re-decide" loop is clear and controllable.
- `execute_action_batch(...)`: executes actions and records terminal progress.
- `build_action_feedback_query(...)`: converts action execution results into feedback text that Dify can understand.

4.4 Box-Selection Layer and Tracking Actions

The box-selection layer is visible in the Dify backend and does not run locally in ROS; it is a vision node in the Dify workflow. The corresponding prompt is at:

When tracking a target, the recommended workflow is as follows:

User requests to track a target

- > Decision layer plans "get the current image, select the target, and start tracking and following"
- > Execution layer first outputs `seewhat()`
- > `llm_agent_node` saves the current image and feeds it back to Dify
- > Box-selection layer reads the image and outputs `bbox_OUT`
- > Execution layer reads `bbox_OUT` and outputs `track_kcf(x1,y1,x2,y2)`
- > `llm_agent_node` calls the KCF tracking service

The box-selection layer output must be strict JSON:

```
{"found":true,"target":"target name","bbox":[x1,y1,x2,y2],"confidence":0.0,"reason":"brief description"}
```

The execution layer only uses the valid bbox given by the box-selection layer. It is forbidden for the execution layer to guess coordinates based on the image itself, which prevents the target box from drifting, boxing the background, or confusing multiple targets of the same type.

```
for round_index in range(1, max_rounds + 1):
    dify_response = self.dify.call_agent(
        request_text,
        robot_feedback=robot_feedback,
        image_paths=feedback_image_paths,
    )
    actions, response_text = self.dify.parse_response(dify_response)
    action_batch, ignored_actions, finish_requested = select_round_action(actions)
    batch_success = self.execute_action_batch(action_batch, executed_actions)
    pending_query, pending_images = self.consume_pending_action_feedback()
    request_text = pending_query or self.build_action_feedback_query(
        action_batch, batch_success
    )
    feedback_image_paths = pending_images
    robot_feedback = True
```

5. Action Execution Feature

The current package does not have a separate action server file. Actions are distributed by the action registry in `llm_agent_node.py` to each controller for execution.

File	Main Function
<code>controllers/motion_controller.py</code>	Chassis forward, backward, left turn, right turn, publishing velocity through <code>/cmd_vel</code>
<code>controllers/navigation_controller.py</code>	Nav2 navigation, recording the current position, canceling the current navigation goal
<code>controllers/ptz_controller.py</code>	Pan-tilt servo reset, nod, shake, and rotation by a specified angle
<code>controllers/follow_line_controller.py</code>	Reads HSV calibration values, performs color line following and obstacle protection
<code>controllers/kcf_tracker.py</code>	Receives the box-selection layer bbox, calls the external KCF tracking service via <code>track_kcf(x1,y1,x2,y2)</code> , subscribes to the tracking box and controls the pan-tilt/chassis to follow

Action strings uniformly use the `action_name(parameters)` format, for example:

```

move_forward(0.2, 2.0)
turn_left(90, 0.5)
navigation(A)
follow_line(red)
seewhat()
observe_surroundings()
track_kcf(120, 80, 360, 300)
finishtask()

```

`execute_action` parses the action name and parameters, looks up the action registry, and calls the corresponding handler function.

```

def execute_action(self, action_str: str) -> bool:
    match = re.fullmatch(r"(\w+)\((.*)\)", action_str)
    action_name, args_str = match.groups()
    args = [arg.strip().strip('"\'') for arg in args_str.split(",") if arg.strip()]

    handler = self.action_registry.get(action_name)
    if handler:
        return handler(*args)

    self.get_logger().error(f"[ACTION] unknown action: '{action_name}'")
    return False

```

6. Preemption and Stop Feature

Since the current entry point is terminal text, preemption no longer relies on voice wake-up. When the user inputs a new instruction during action execution, the core node increments `current_command_id`, and the old worker thread determines that it has expired through `should_interrupt()` and exits the current flow as soon as possible.

6.1 New Instruction Preemption

If the robot is executing an action, `text_callback` calls `cancel_current_action()`:

1. Cancels the current Nav2 navigation goal.
2. Stops the line-following controller.
3. Stops KCF tracking.
4. Publishes a stop velocity.
5. Recenters the pan-tilt if needed.

```

def cancel_current_action(self, recenter_camera: bool = False):
    self.navigation.cancel_active_goal()
    self.follow_line_controller.stop()
    self.execute_stop(recenter_camera=recenter_camera)

```


6.2 Worker Thread Expiry Determination

Each user instruction has an independent `command_id`. When a new instruction arrives, the old thread's `command_id` does not match `current_command_id`, so `should_interrupt()` returns `True`, and the old thread stops requesting Dify or executing actions.

```
def should_interrupt(self) -> bool:
    state = getattr(self.worker_context, "state", None)
    if state is None:
        return False

    with self.command_lock:
        return state.command_id != self.current_command_id
```

6.3 Stop Action

The `stop()` action stops KCF following and the line-following controller, and optionally recenters the pan-tilt based on the parameter.

```
def execute_stop(self, recenter_camera: bool = True) -> bool:
    self.kcf.stop_active_kcf_tracking()
    self.follow_line_controller.stop()
    if self._parse_bool_flag(recenter_camera, default=True):
        self.ptz.recenter_after_stop()
    return True
```

Chassis-type actions, line following, observation, and KCF following also periodically check the preemption status internally; after receiving a new instruction, they stop the current action as much as possible and hand control to the latest instruction.